

¿Qué es una función Async-Signal-Safe?

Es una función que POSIX garantiza que se puede invocar de forma segura **dentro de un manejador de señales** sin causar corrupción de memoria o interbloqueos (**deadlocks**).

El peligro: Reentrancia y Locks

Funciones como `printf()` o `malloc()` usan **buffers y locks internos**. Si una señal interrumpe a `printf()` a mitad de ejecución y el manejador vuelve a llamar a `printf()`, el proceso se colgará esperando su propio lock.

Ejemplos clave (POSIX)

- **Seguras:** `write()`, `read()`, `_exit()`, `sigaction()`, `pause()`.
- **NO seguras:** `printf()`, `malloc()`, `free()`, `exit()`, `sprintf()`.

- Como dijimos previamente, `wait()` espera a que un hijo cambie de estado.

- Como dijimos previamente, `wait()` espera a que un hijo cambie de estado.
- Pero, ¿cómo funciona este mecanismo de espera?

- Como dijimos previamente, `wait()` espera a que un hijo cambie de estado.
- Pero, ¿cómo funciona este mecanismo de espera?
- Cuando un proceso hijo termina su ejecución, envía la señal SIGCHLD y el padre la recibe.

- Como dijimos previamente, `wait()` espera a que un hijo cambie de estado.
- Pero, ¿cómo funciona este mecanismo de espera?
- Cuando un proceso hijo termina su ejecución, envía la señal `SIGCHLD` y el padre la recibe.
- No solamente se envía en este caso, sino también cuando ocurre un cambio de estado: que un hijo terminó, que el hijo fue frenado por una señal, o el hijo fue reanudado por una señal.

Capabilities

- En Linux se distinguen procesos con permisos privilegiados (root) y no privilegiados.

Capabilities

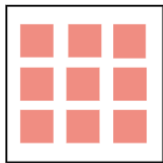
- En Linux se distinguen procesos con permisos privilegiados (root) y no privilegiados.
- Un proceso no root no puede enviar señales a procesos root

Capabilities

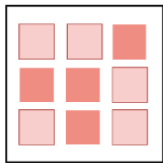
- En Linux se distinguen procesos con permisos privilegiados (root) y no privilegiados.
- Un proceso no root no puede enviar señales a procesos root
- Sin embargo, en Linux es posible permitir este envío, ya que divide los privilegios tradicionalmente asociados con root en distintas unidades llamadas **capabilities**, que pueden ser habilitadas o deshabilitadas.

Capabilities

- En Linux se distinguen procesos con permisos privilegiados (root) y no privilegiados.
- Un proceso no root no puede enviar señales a procesos root
- Sin embargo, en Linux es posible permitir este envío, ya que divide los privilegios tradicionalmente asociados con root en distintas unidades llamadas **capabilities**, que pueden ser habilitadas o deshabilitadas.
- Se puede usar `setcap` para cambiar las **capabilities**, CAP_KILL se denomina a la capacidad para enviar señales a cualquier proceso,



full capabilities



specific capabilities

Concepto: El "Filtro" del Kernel

La **máscara de señales** es un mapa de bits (**bitmask**) almacenado dentro de la estructura del proceso (`task_struct`) en el kernel.

Concepto: El "Filtro" del Kernel

La **máscara de señales** es un mapa de bits (**bitmask**) almacenado dentro de la estructura del proceso (`task_struct`) en el kernel.

- **Bit en 0 (Desbloqueada):** La señal se entrega de inmediato al proceso.
- **Bit en 1 (Bloqueada / Enmascarada):** La señal no se pierde; queda retenida como **pendiente** hasta que se vuelva a desbloquear.

Concepto: El "Filtro" del Kernel

La **máscara de señales** es un mapa de bits (**bitmask**) almacenado dentro de la estructura del proceso (`task_struct`) en el kernel.

- **Bit en 0 (Desbloqueada):** La señal se entrega de inmediato al proceso.
- **Bit en 1 (Bloqueada / Enmascarada):** La señal no se pierde; queda retenida como **pendiente** hasta que se vuelva a desbloquear.

Representación en Bits

SIGKILL (9)	SIGALRM (14)	SIGTERM (15)	SIGINT (2)
0	0	0	1 (<i>Bloqueada</i>)

La llamada al sistema `sigsuspend`

```
int sigsuspend(const sigset_t *mask);
```

La llamada al sistema sigsuspend

```
int sigsuspend(const sigset_t *mask);
```

Comportamiento Atómico

Realiza las siguientes acciones en un **único paso indivisible** dentro del kernel:

1. Reemplaza temporalmente la máscara del proceso por `mask` (donde la señal esperada **no** debe estar bloqueada).
2. Pone a dormir el proceso a la espera de una señal que ejecute un manejador o lo termine.

API - S.O.

Sincronización Segura (sigsuspend)

La llamada al sistema sigsuspend

```
int sigsuspend(const sigset_t *mask);
```

Comportamiento Atómico

Realiza las siguientes acciones en un **único paso indivisible** dentro del kernel:

1. Reemplaza temporalmente la máscara del proceso por `mask` (donde la señal esperada **no** debe estar bloqueada).
2. Pone a dormir el proceso a la espera de una señal que ejecute un manejador o lo termine.

Retorno y Restauración

- Cuando el manejador de la señal termina, `sigsuspend()` retorna `-1` y asigna `errno = EINTR`.
- Al retornar, el kernel **restaura automáticamente** la máscara de

Funciones para Modificar el Tipo `sigset_t`

Antes de cambiar la máscara del proceso, debemos construir el conjunto de señales en memoria:

Funciones para Modificar el Tipo sigset_t

Antes de cambiar la máscara del proceso, debemos construir el conjunto de señales en memoria:

- `sigemptyset(&set)`: Vacía el conjunto (pone todos los bits en 0).
¡Siempre inicializar con esto!

Funciones para Modificar el Tipo sigset_t

Antes de cambiar la máscara del proceso, debemos construir el conjunto de señales en memoria:

- `sigemptyset(&set)`: Vacía el conjunto (pone todos los bits en 0).
¡Siempre inicializar con esto!
- `sigfillset(&set)`: Incluye todas las señales en el conjunto (bits en 1).

Funciones para Modificar el Tipo sigset_t

Antes de cambiar la máscara del proceso, debemos construir el conjunto de señales en memoria:

- `sigemptyset(&set)`: Vacía el conjunto (pone todos los bits en 0).
¡Siempre inicializar con esto!
- `sigfillset(&set)`: Incluye todas las señales en el conjunto (bits en 1).
- `sigaddset(&set, SIGINT)`: Añade una señal específica al conjunto.

Funciones para Modificar el Tipo sigset_t

Antes de cambiar la máscara del proceso, debemos construir el conjunto de señales en memoria:

- `sigemptyset(&set)`: Vacía el conjunto (pone todos los bits en 0).
¡Siempre inicializar con esto!
- `sigfillset(&set)`: Incluye todas las señales en el conjunto (bits en 1).
- `sigaddset(&set, SIGINT)`: Añade una señal específica al conjunto.
- `sigdelset(&set, SIGINT)`: Remueve una señal específica del conjunto.

Funciones para Modificar el Tipo sigset_t

Antes de cambiar la máscara del proceso, debemos construir el conjunto de señales en memoria:

- `sigemptyset(&set)`: Vacía el conjunto (pone todos los bits en 0).
¡Siempre inicializar con esto!
- `sigfillset(&set)`: Incluye todas las señales en el conjunto (bits en 1).
- `sigaddset(&set, SIGINT)`: Añade una señal específica al conjunto.
- `sigdelset(&set, SIGINT)`: Remueve una señal específica del conjunto.
- `sigismember(&set, SIGINT)`: Verifica si la señal está en el conjunto (retorna 1 o 0).

API - S.O.

Aplicar la Máscara al Proceso (sigprocmask)

La llamada al sistema sigprocmask

```
int sigprocmask(int how, const sigset_t *set, sigset_t *oldset);
```

La llamada al sistema sigprocmask

```
int sigprocmask(int how, const sigset_t *set, sigset_t *oldset);
```

Modos de operación (how)

- SIG_BLOCK: Bloquea las señales pasadas en set (las añade a la máscara actual).

La llamada al sistema sigprocmask

```
int sigprocmask(int how, const sigset_t *set, sigset_t *oldset);
```

Modos de operación (how)

- SIG_BLOCK: Bloquea las señales pasadas en set (las añade a la máscara actual).
- SIG_UNBLOCK: Desbloquea las señales pasadas en set (las quita de la máscara actual).

La llamada al sistema sigprocmask

```
int sigprocmask(int how, const sigset_t *set, sigset_t *oldset);
```

Modos de operación (how)

- SIG_BLOCK: Bloquea las señales pasadas en set (las añade a la máscara actual).
- SIG_UNBLOCK: Desbloquea las señales pasadas en set (las quita de la máscara actual).
- SIG_SETMASK: Reemplaza la máscara actual por completo con set.

API - S.O.

Aplicar la Máscara al Proceso (sigprocmask)

La llamada al sistema sigprocmask

```
int sigprocmask(int how, const sigset_t *set, sigset_t *oldset);
```

Modos de operación (how)

- SIG_BLOCK: Bloquea las señales pasadas en set (las añade a la máscara actual).
- SIG_UNBLOCK: Desbloquea las señales pasadas en set (las quita de la máscara actual).
- SIG_SETMASK: Reemplaza la máscara actual por completo con set.

Parámetro oldset

Si no es NULL, el kernel guarda allí la máscara que estaba activa antes del cambio (útil para restaurarla después).